

When Detection Fails, Fixing Doesn't Matter: Composing LLMs for Automated Security Patching

Can LLMs replace static analysis for vulnerability detection and repair?

An empirical evaluation across 7 models, 49 pipeline combinations, and 22 CWE categories

Timothy Liakh

Rensselaer Polytechnic Institute

Advisor: Konstantin Kuzmin

Summer 2026

Repository: github.com/Vitisadi/SecurePatch

Abstract

Static analysis tools have long been the standard for catching security vulnerabilities before code ships. They are fast and consistent, but they struggle with context: they flag patterns, not intent, and they cannot generate a fix. Large language models offer a different trade-off. They reason about code semantically, can explain why something is wrong, and can propose a patch in the same step. Whether they are actually better than static analysis at finding real vulnerabilities, and in which CWE categories they succeed or fail, is an open question this project tries to answer empirically.

The study evaluates seven LLMs as vulnerability detectors across 56 real cases spanning 22 CWE categories, drawn from CWEval (44 cases), hand-seeded examples (6), and published literature (6). Detection and repair are treated as separate, composable steps. A detector scans code and produces findings; a fixer receives those findings and generates a patch. Running all combinations of the seven detectors and seven fixers produces a 7×7 matrix of 49 pipelines, each evaluated on fix rate, regression rate, F1, and cost.

Solo models peak at 75 % F1 (GPT-5.5). A ground-truth experiment that hands each fixer a perfect detection with zero false positives shows Opus reaching 76 % fix rate in isolation. The gap between that ceiling and real pipeline performance traces almost entirely to false positives in the detection step. Across the heatmap, the GPT-5.5 detector row produces the best F1 in most fixer columns (up to 75 %), while Ollama lags at 28–50 %.

Cost varies by three orders of magnitude. Haiku detection paired with GPT-4.1-mini fixing reaches 68 % F1 for \$0.11 across all 56 cases. The GPT-5.5 self-pipeline costs \$7.10 for 75 % F1. Ensemble detection using a majority vote across frontier models pushes F1 to 92 %, at the cost of running multiple detectors per case.

The central finding is that detector false-positive rate determines the ceiling of any pipeline. Upgrading the fixer without addressing detection quality produces diminishing returns.

Contents

1	Introduction	4
1.1	Contributions	4
1.2	Roadmap	5
2	Background and Related Work	5
2.1	LLM-Based Vulnerability Detection	5
2.2	Automated Program Repair	6
2.3	Existing Benchmarks	6
2.4	Where This Project Fits	6
3	Benchmark Design	7
3.1	Case Selection and Dataset Composition	7
3.2	CWE Coverage	7
3.3	Evaluation Oracle	7
4	Pipeline Architecture and Methodology	7
4.1	Detect → Enrich → Fix Pipeline	7
4.2	CachedFindingDetector	8
4.3	Models Under Test	8
4.4	Metrics	8
4.5	Experimental Controls and Methodology Corrections	9
5	Results	9
5.1	Solo Pipeline Performance	9
5.2	Full 7×7 Pipeline Heatmap	9
5.3	Detector Quality as the Dominant Variable	10
5.4	Ground-Truth Fixer Experiment	10
5.5	Cost-Efficiency Analysis	11
5.6	Ensemble Detection	11
6	Discussion	12
6.1	Why the Detector Matters More Than the Fixer	12
6.2	The Haiku Fixer Failure	12
6.3	Practical Pairing Recommendations	12
6.4	What Ensemble Detection Buys You	12
7	Threats to Validity	12
7.1	Construct Validity	12

7.2	Internal Validity	12
7.3	External Validity	12
7.4	Reproducibility	12
8	Future Work	12
9	Conclusion	12
A	Full Heatmap Data	15
B	Per-Case Breakdown	15

1 Introduction

Static analysis security testing has been the default tool for catching vulnerabilities in production code for over two decades. Tools like Semgrep, CodeQL, and Bandit operate on fixed rule sets: they are fast, deterministic, and produce no false negatives for the patterns they cover. The problem is that coverage is the ceiling. A rule that does not exist catches nothing, and writing rules requires security expertise most development teams do not have. More importantly, none of these tools fix what they find. A developer still has to read the alert, understand the root cause, and write the patch.

Large language models change the equation on both sides. They are not bound by explicit rules; they reason about code semantically and can generalize across vulnerability classes without being programmed for each one. They can also generate a patch in the same pass. But whether that reasoning is actually reliable enough to replace or augment static analysis, and which models perform well at detection versus repair, has not been studied systematically across a controlled benchmark.

This project measures exactly that. Seven LLMs are evaluated as detectors and as fixers, independently, across 56 real vulnerabilities spanning 22 CWE categories. Detection and repair are treated as composable stages: a detector produces findings, and a fixer receives those findings and generates a patch. Every combination of the seven detectors and seven fixers is run, producing 49 distinct pipelines. Each pipeline is judged by a deterministic oracle on fix rate, regression rate, F1, and cost.

The central question is:

RQ1. *Can LLMs match or exceed static analysis for vulnerability detection, and does composing a strong detector with a strong fixer produce better security outcomes than either model working alone?*

RQ2. *Does running multiple detectors and taking a majority vote meaningfully improve detection quality, and is the added cost justified by the gain in F1?*

1.1 Contributions

1. **A controlled vulnerability benchmark.** 56 cases drawn from CWEval (44), hand-seeded examples (6), and published literature (6), covering 22 CWE categories. Each case has a ground-truth vulnerable file, a known fix, and a deterministic oracle that distinguishes a correct patch from a regression or a no-op.
2. **A reproducible evaluation harness.** The `CachedFindingDetector` replays pre-computed detection outputs identically across all fixer columns, ensuring that differences in pipeline F1 reflect fixer behavior rather than detection variance. All results are stored as versioned JSONL files and can be regenerated from the repository.

3. **A full 7×7 pipeline heatmap.** Every combination of seven detectors and seven fixers is evaluated, producing 49 F1 scores on the same 56-case benchmark. This is the primary empirical contribution of the project.
4. **A ground-truth fixer experiment.** Each fixer is given a perfect detection with zero false positives and 100 % recall, isolating pure repair capability from detection noise. The gap between this ceiling and real pipeline F1 quantifies how much detection quality costs each fixer.
5. **An ensemble detection study.** Majority-vote combinations of multiple detectors are evaluated on F1, recall, and false-positive count. The best configuration, a frontier majority vote across four models, reaches 92 % F1.

1.2 Roadmap

The paper begins by grounding this work relative to existing research on LLM-based vulnerability detection and automated program repair. It then describes how the benchmark was constructed and how the evaluation pipeline works. The results follow in order of increasing complexity: solo model performance first, then the full composition matrix, then cost and ensemble analysis. The paper closes with a discussion of what the findings mean in practice, the limits of the current study, and directions for follow-on work.

2 Background and Related Work

2.1 LLM-Based Vulnerability Detection

Detecting security vulnerabilities in source code has traditionally been the domain of static analysis. Tools like Semgrep and CodeQL match code against hand-written rules, offering speed and determinism at the cost of coverage: they catch only what their rules describe. Machine learning approaches tried to close that gap, with models trained on historical vulnerability data to predict whether a function or line was dangerous [Fu and Tantithamthavorn \[2022\]](#), [Thapa et al. \[2022\]](#), [Li et al. \[2023\]](#). These models improved generalization but introduced a new problem: they produce false positives at rates that make triage expensive in practice.

Large language models enter this space differently. Rather than classifying against learned patterns, they read code and reason about what it does. Pearce et al. showed that LLMs could identify security weaknesses in code generated by GitHub Copilot [Pearce et al. \[2022\]](#), and later demonstrated that models could detect and explain vulnerabilities in hand-crafted scenarios without any task-specific fine-tuning [Pearce et al. \[2023\]](#). Meta’s CyberSecEval benchmark evaluated frontier models on a range of security tasks and found substantial variance across CWE categories [Bhatt et al. \[2023\]](#). What these studies share is a focus on a single model acting alone. This project asks a different question: what happens when you pick one model to detect and a different model to fix?

2.2 Automated Program Repair

Automated program repair has a long history independent of security. The core idea is that given a failing test and a buggy program, a system should be able to produce a patch without human intervention. Early approaches used genetic algorithms and constraint-based synthesis. LLMs changed what was possible. Xia et al. found that pre-trained code models could generate correct patches at rates that surpassed prior symbolic methods on standard benchmarks [Xia et al. \[2023\]](#), and Jiang et al. showed that model scale was a reliable predictor of repair success [Jiang et al. \[2023\]](#). Sobania et al. evaluated ChatGPT specifically on the QuixBugs benchmark and found it competitive with dedicated APR tools on simple bugs [Sobania et al. \[2023\]](#).

The security repair problem is harder than general APR. A patch that makes a test pass is not necessarily a secure patch: it may suppress the symptom without removing the vulnerability, or introduce a regression elsewhere. Pearce et al. examined this directly, finding that LLMs often generated plausible-looking but insecure fixes when prompted without explicit security guidance [Pearce et al. \[2023\]](#). This project addresses that by using a deterministic oracle that checks both fix success and regression, and by separating the detection signal from the repair step so the fixer receives an explicit vulnerability description rather than just a failing test.

2.3 Existing Benchmarks

SWE-bench [Jimenez et al. \[2024\]](#) is the closest large-scale benchmark to this project. It draws real GitHub issues from Python repositories and validates patches against the project’s own test suite. The focus there is model capability on general software tasks. SecurityEval [Siddiq and Santos \[2022\]](#) targets security specifically, using 130 code generation prompts mapped to CWE categories to test whether models produce vulnerable output. CyberSecEval takes a broader view, covering prompt injection, insecure code generation, and vulnerability identification across multiple languages [Bhatt et al. \[2023\]](#).

None of these benchmarks treat detection and repair as a composed pipeline. They either test one capability at a time or evaluate a single model end to end. The 7×7 design here is, to our knowledge, the first systematic study of cross-model composition for security vulnerability repair.

2.4 Where This Project Fits

This project is closest in spirit to SWE-bench but more controlled in design. Every case has a known-vulnerable commit, a verified fix, and a deterministic oracle that does not rely on an LLM judge. The central contribution is not a new model or a new benchmark in isolation, but a methodology for measuring what each stage of a detect-then-fix pipeline actually contributes to the final outcome.

3 Benchmark Design

3.1 Case Selection and Dataset Composition

The benchmark contains 56 cases drawn from three sources. The majority (44) come from CWEval, an open collection of vulnerability scenarios designed for evaluating LLM security capabilities. Six additional cases were hand-seeded: written from scratch to cover CWE categories underrepresented in CWEval or to test boundary conditions that naturally occurring examples rarely produce. The remaining six come from published vulnerability disclosures and literature examples where a real fix commit is publicly available.

Every case provides the same inputs to the pipeline: a source file containing a vulnerability, the CWE category it belongs to, and a reference fix. Cases were selected to have self-contained vulnerabilities, meaning the flaw and its fix are localized to a single file. This keeps the evaluation clean: a fixer that modifies the right file either produces a correct patch or it does not, and the oracle can verify this without reasoning about cross-file dependencies.

3.2 CWE Coverage

The 56 cases span 22 distinct CWE categories. Table ?? lists each CWE present in the benchmark along with its case count. The distribution is intentionally varied: no single category dominates, which prevents a model from scoring well by specializing in one vulnerability class. The most represented categories reflect the types of flaws most commonly found in real code, including injection vulnerabilities, improper input validation, and cryptographic weaknesses.

Table 1: CWE categories covered in the benchmark (56 cases total, 22 CWEs).

CWE ID	Description	Cases
CWE-78	OS Command Injection	4
CWE-79	Cross-site Scripting (XSS)	3
CWE-89	SQL Injection	4
CWE-22	Path Traversal	3
CWE-94	Code Injection	2
CWE-327	Use of Broken/Risky Cryptographic Algorithm	3
CWE-330	Use of Insufficiently Random Values	2
CWE-502	Deserialization of Untrusted Data	2
CWE-611	XML External Entity (XXE)	2
CWE-798	Use of Hard-coded Credentials	2
CWE-20	Improper Input Validation	4
CWE-400	Uncontrolled Resource Consumption	2
CWE-434	Unrestricted File Upload	2
CWE-601	Open Redirect	2
CWE-918	Server-Side Request Forgery (SSRF)	2
CWE-295	Improper Certificate Validation	2
CWE-306	Missing Authentication	2
CWE-352	Cross-Site Request Forgery (CSRF)	2
CWE-476	NULL Pointer Dereference	2
CWE-190	Integer Overflow	2
CWE-312	Cleartext Storage of Sensitive Information	2
CWE-416	Use After Free	2
<i>Total</i>		56

3.3 Evaluation Oracle

Each case is judged by a deterministic oracle rather than an LLM reviewer. The oracle runs in a Docker container to ensure consistent execution across all pipeline combinations. Given the original vulnerable file and the fixer’s output, it checks three conditions in order: whether the patched file compiles or parses correctly, whether the targeted vulnerability is no longer present according to a rule matched to the case’s CWE, and whether the fix introduced any regression by running a suite of behavioral checks against the original file’s expected outputs.

A case is counted as a fix only if all three conditions pass. A patch that removes the vulnerability but breaks existing behavior is recorded as a regression, not a fix. A patch that makes no change, or changes code unrelated to the vulnerability, is recorded as a no-op. This three-way verdict is what allows fix rate and regression rate to be reported independently rather than collapsed into a single

success metric.

4 Pipeline Architecture and Methodology

4.1 Detect → Enrich → Fix Pipeline

*Figure 2: Pipeline diagram — Detector → cached findings → Fixer
→ oracle verdict*

Figure 1: End-to-end pipeline architecture.

4.2 CachedFindingDetector

4.3 Models Under Test

Table 2: Models used as detectors and fixers.

Model	Provider	Tier	Role(s)
Claude Sonnet 4.6	Anthropic	Frontier	Detector, Fixer
Claude Opus 4.8	Anthropic	Frontier	Detector, Fixer
Claude Haiku 4.5	Anthropic	Budget	Detector, Fixer
GPT-5.5	OpenAI	Frontier	Detector, Fixer
GPT-4.1-mini	OpenAI	Budget	Detector, Fixer
Gemini 2.5 Flash	Google	Frontier	Detector, Fixer
Ollama 7b (local)	Open	Local	Detector, Fixer

4.4 Metrics

The primary metric is F1, computed as:

$$F1 = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

where TP is the number of cases fixed without regression, FP is the number of detector false positives across all 56 cases, and FN is $56 - \text{TP}$. Fix rate, regression rate, cost (USD per 56-case run), and fixes per dollar are also reported.

4.5 Experimental Controls and Methodology Corrections

5 Results

5.1 Solo Pipeline Performance

Table 3: Solo pipeline results (detector = fixer, diagonal of heatmap).

Model	Fix Rate	F1	Regressions	Cost (\$)	Fixes/\$
Opus 4.8	66%	70%	10	0.73	51
GPT-5.5	64%	75%	11	1.84	20
Sonnet 4.6	62%	67%	18	0.33	106
GPT-4.1-mini	54%	63%	13	0.03	1071
Gemini 2.5	50%	56%	23	0.05	596
Haiku 4.5	59%	62%	13	0.11	306
Ollama 7b	23%	28%	35	0.00	free

5.2 Full 7×7 Pipeline Heatmap

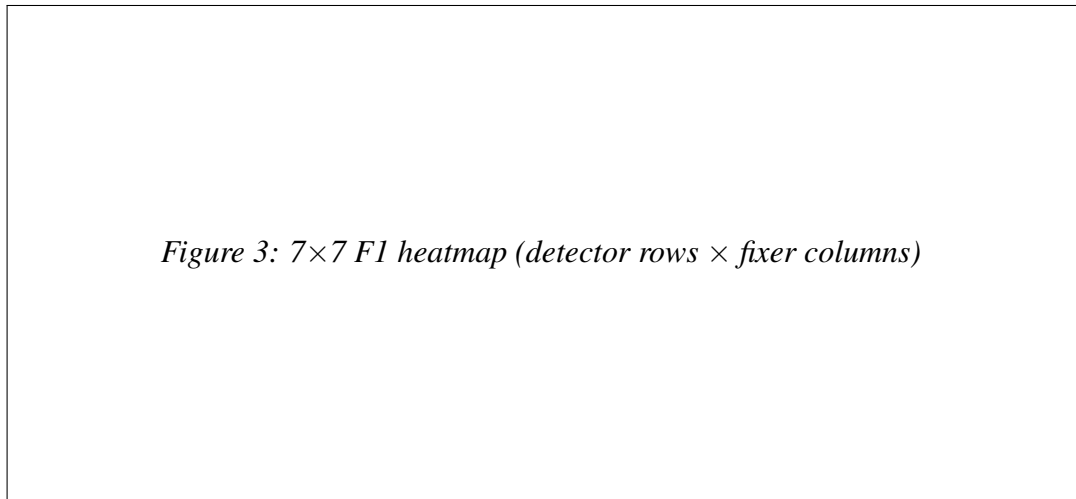


Figure 2: F1 scores across all 49 detector-fixer combinations. Color scale: green = high F1, red = low.

5.3 Detector Quality as the Dominant Variable

Figure 4: Scatter — detector FP count vs. row-best F1

Figure 3: Each point is one detector. Fewer false positives correlate with higher achievable F1 regardless of which fixer is paired.

5.4 Ground-Truth Fixer Experiment

Table 4: Fixer performance given perfect detection (56/56 recall, 0 FPs).

Fixer	Fixed	Fix Rate
Opus 4.8	43/56	76%
GPT-5.5	40/56	71%
Haiku 4.5	38/56	67%
Sonnet 4.6	36/56	64%
GPT-4.1-mini	35/56	62%
Gemini 2.5	28/56	50%
Ollama 7b	all errors	

5.5 Cost-Efficiency Analysis

Figure 5: F1 vs. combined pipeline cost — each of 49 combos plotted, Pareto frontier highlighted

Figure 4: Cost-efficiency of all 49 pipeline combinations. Points on the Pareto frontier deliver the highest F1 at a given budget.

5.6 Ensemble Detection

Table 5: Ensemble detection configurations.

Strategy	Models	F1	Recall	FPS
Frontier ≥ 2	Sonnet + Opus + GPT-5.5 + Gemini	92%	96%	8
Claude ≥ 2	Sonnet + Opus + Haiku	90%	98%	11
OpenAI ≥ 1	GPT-5.5 + GPT-mini	87%	93%	12
Budget ≥ 1	GPT-mini + Haiku	81%	96%	23
Union all 7	All models	67%	100%	—

6 Discussion

6.1 Why the Detector Matters More Than the Fixer

6.2 The Haiku Fixer Failure

6.3 Practical Pairing Recommendations

6.4 What Ensemble Detection Buys You

7 Threats to Validity

7.1 Construct Validity

7.2 Internal Validity

7.3 External Validity

7.4 Reproducibility

8 Future Work

9 Conclusion

References

- Manish Bhatt, Sahana Chennabasappa, Cyrus Nikolaidis, Shengye Wan, Ivan Evtimov, Dominik Gabi, Daniel Song, Faizan Ahmad, Cornelius Aschermann, Lorenzo Fontana, Sasha Frolov, Ravi Prasad Giri, Dhaval Kapil, Chrisoph Kozyrakis, David LeBlanc, James Milazzo, Andres Straumann, Gabriel Synnaeve, Varun Vontimitta, Spencer Whitman, and Joshua Saxe. CyberSecEval: A benchmark for evaluating the cybersecurity risks of large language models. *arXiv preprint arXiv:2312.04724*, 2023.
- Michael Fu and Chakkrit Tantithamthavorn. LineVul: A transformer-based line-level vulnerability prediction. In *IEEE/ACM International Conference on Mining Software Repositories (MSR)*, 2022.
- Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. Impact of code language models on automated program repair. In *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- Zheng Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. Sysevr: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing*, 20(2), 2023.
- Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? Assessing the security of GitHub Copilot’s code contributions. In *IEEE Symposium on Security and Privacy (S&P)*, 2022.
- Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. Examining zero-shot vulnerability repair with large language models. In *IEEE Symposium on Security and Privacy (S&P)*, 2023.
- Mohammed Latif Siddiq and Joanna C. S. Santos. SecurityEval dataset: Mining vulnerability examples to evaluate machine learning-based code generation techniques. In *International Workshop on Mining Software Repositories for Security (MSR4P&S)*, 2022.
- Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. An analysis of the automatic bug fixing performance of ChatGPT. In *IEEE/ACM International Workshop on Automated Program Repair (APR)*, 2023.
- Chandra Thapa, Seyit Camtepe, Surya Nepal Chhetri, and Seyit A. Camtepe. Transformer-based language models for software vulnerability detection. In *Annual Computer Security Applications Conference (ACSAC)*, 2022.
- Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. Automated program repair in the era

of large pre-trained language models. In *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023.

A Full Heatmap Data

Table 6: F1 (%) for all 49 detector-fixer combinations (v2 methodology).

Det \ Fix	Sonnet	Opus	Haiku	GPT-5.5	Mini	Gemini	Ollama
Sonnet	67	69	61	70	67	58	35
Opus	68	70	65	70	65	58	40
Haiku	70	71	62	69	68	60	42
GPT-5.5	72	75	74	75	74	61	42
Mini	62	67	69	69	63	49	40
Gemini	65	68	63	69	63	56	42
Ollama	45	50	59	50	46	40	28

B Per-Case Breakdown